Proceedings of Machine Learning Research 329:1–23, 2026 Conformal and Probabilistic Prediction with Applications

Continuous Assurance of Digital Assets via Conformal Prediction over Polytope-Defined Operating Regions

Hee Jong Park HEE.JONG.PARK@DNV.COM
Kristoffer Skare KRISTOFFER.SKARE@DNV.COM
Cesar Augusto Ramos de Carvalho CESARDECARVALHO@GMAIL.COM
DNV AS, Norway

Claas Rostock CLAAS.ROSTOCK@DNV.COM
DNV SE, Germany

Editor: Ernst Ahlberg, Ulf Johansson, Henrik Boström, Alberto Carlevaro, Johan Hallberg Szabadváry and Lars Carlsson

Abstract

Continuous assurance is a process for applying an agile, modular and iterative process to re-assure and verify changes in digital assets, such as simulation models, against a set of predefined requirements. In the recent years, several verification methods have been developed that can be used in the continuous assurance context to achieve statistically sufficient coverage of simulation space. Instead of exhaustively traversing the simulation search space to find a solution that falsifies the requirements, these methods partition the space into smaller and interpretable sub-regions. Statistical methods for computing prediction intervals for the subsequent sample provides a basis for determining whether simulation parameter sampled from these sub-regions are likely to satisfy or violate the requirement contract. In this work, we apply such techniques to test a converter controller model used in a grid-forming wind turbine, within the context of a continuous assurance cycle. Additionally, we extend the search space partitioning method from existing literature by creating regions of convex polytopes near class boundaries. These regions are linear in the optimization problem, with the aim to reduce the regions of uncertain areas during classification. We further demonstrate the partitioning method with an additional set of benchmarks including the mountain car and F-16 GCAS simulation model.

Keywords: Continuous assurance, Conformal prediction, Gaussian process, Simulation, Design of Experiments, Signal Temporal Logic, Virtual Testing

1. Introduction

Modern complex systems increasingly rely on software-determined behavior, making assurance tasks more challenging across a wide range of engineering fields (NASA, 2026). These systems are made to dynamically adapt to changing environments and their behaviors can be updated via software updates on-demand (Asaadi et al., 2020). Additionally, the growth in the size and complexity of software systems further challenges the traditional assurance process. This drives the need for *continuous assurance* that lifts the classical assurance process to be more modular, continuous and repeatable (Wei et al., 2024). In each continuous assurance cycle, the updated system's behavior is verified to ensure it still conforms to the system's specification, which is defined in machine readable form.

Modern complex systems often consist of many vendor-supplied *black-box* components, for which visibility into the software or hardware are often limited due to Intellectual Property (IP) constraints. Consequently, verification methods for such components are typically restricted to monitoring the system's behavior through the input-output interactions and feedback. For example, machine learning techniques, such as surrogate modeling are widely used techniques to approximate the behavior of black-box models and are commonly used in statistical verification approaches (Torben et al., 2023; Zhang and Arcaini, 2021) via *falsification* of the system's specifications. Nevertheless, while the quality of falsification depends on a ground basis for assessing the level of confidence in the result, this assessment is often challenging due to unknown uncertainties in both the inputs and black-box nature of system components.

When creating surrogate models, it is often necessary to use techniques to quantify the model's statistical guarantees, as a mean to assess reliability of the overall verification results. The recent works in (Qin et al., 2024; Pedrielli et al., 2023) have proposed methods for estimating the quality of surrogate models. In particular, instead of creating a surrogate on the black-box model's global parameter space, their methods create a collection of smaller surrogates that each approximates sub-regions of the parameter space. The rationale behind such method is twofold: on one hand, each surrogate model can better approximate sub-regions of relatively simple surface. On the other hand, the approach is more scalable for models such as Gaussian Process (GP), which has cubic complexity with respect to the number of training samples. Nevertheless, both methods consider only hyper-rectangular sub-regions, which are often ill-suited for classifying regions with irregular boundaries.

In this paper, we extend the approaches from (Qin et al., 2024; Pedrielli et al., 2023) for use in the statistical verification of a digital asset. We use a converter controller in a Grid-Forming Wind Turbine Generator (WTG) as a use-case scenario in the context of continuous assurance process. Rather than simply dividing the region into axis-aligned hyperrectangles, we allow an option to partition the space into convex polytopes. The idea behind this approach is to minimize the volume of uncertain as well as misclassified regions near decision boundaries between class regions. The main contributions of this paper are summarized as follows:

1. An automated testing tool that can be integrated into the testing stage of a continuous assurance framework to assist in identifying irregularities resulting from changes in software, models, or larger parts of the system components.

2. An extension to the partitioning strategy for exploring a parameter search space. Specifically, our approach to classifying the search region builds on (Qin et al., 2024) that utilizes a conformal prediction framework to determine whether samples from the region conforms to a user-defined formal contract (Benveniste et al., 2018). We introduce two different partitioning strategies that are applied when the volume of a partitioning regions falls below a user-defined threshold.

3. Region partitioning strategy based on conformal prediction, where the the prediction interval is adaptively adjusted according to the variance of predictor, modeled as Gaussian Process in our use-case.

4. A case study to evaluate the effectiveness of the partitioning strategy using a series of existing experiments for comparison. Additionally, an additional experiment based on the converter controller of the grid-forming wind turbine generator.

The rest of the paper is organized as follows: Section 2, contains a literature review on continuous assurance and related methodologies that serves as enabling technologies. In Section 3, we introduce an overview of a grid-forming wind turbine generator in the context of a continuous assurance concept. Backgrounds on methods are presented in Section 4. Region partitioning and classification method based on conformal prediction are described in Section 5. Experimental results including the converter controller for the wind turbine generator are provided in Section 6. Finally conclusion and future work are presented in Section 7.

2. Literature Review

In the context of continuous assurance frameworks and high-level analysis of related assurance workflows, several relevant studies can be identified. In (Hake et al., 2021), the authors introduced an integration and verification framework based on *Contract-Based Design* (CBD) concepts where contracts are used in various phases of the development process. Their framework outlines how CBDs are verified and applied throughout the life-cycle of a maritime collision avoidance system. Their framework, however, lacks the practical methods for efficiently exploring the (parameter) search space of models under contracts, particularly in the presence of uncertainties arising from black box models. A work in (Warg et al., 2019) proposes an incremental and modular way to create assurance cases and illustrates how this process can be achieved using existing methods including CBD, and modular and continuous assessment. Again, there is no concrete example though how to automate and conduct the target system testing according to defined contracts. Authors in (Schleiss et al., 2022) highlight the importance of continuous assurance in automotive industry, with a particular focus on run-time system monitoring and uncertainty quantification. In their framework, system design is updated through a continuous feedback loop based on run-time data, which cannot be easily quantified during the design time. Yet, their framework does not address systematic strategies for measuring uncertainties by exploring system parameters.

Design of Experiments (DoE) and smart testing methodologies employ statistical techniques to efficiently plan and execute tests. In (Torben et al., 2023), authors introduced a framework to define and verify requirements of a collision avoidance system for an autonomous vessel using Bayesian interference and Signal Temporal Logic (STL) as a robustness metric for falsifying the requirements. Authors in (Qin et al., 2024) introduced a statistical verification framework using conformal prediction that partitions a global parameter space into many sub-regions until each region can be classified as *safe* or *unsafe* according to the STL specification with *marginal coverage guarantees*. A similar approach is introduced in (Pedrielli et al., 2023) with more focus on finite time guarantees of the partitioning algorithm via user-defined evaluation budget. Unlike (Qin et al., 2024), (Pedrielli et al., 2023) uses biased sampling approach for training the Gaussian Process (GP) and employs Monte-Carlo method for estimating minimum and maximum quantiles from the predictive variance of the GP. In this work, we extend the idea of parameter space exploration technique using conformal prediction inspired by (Qin et al., 2024) and generate

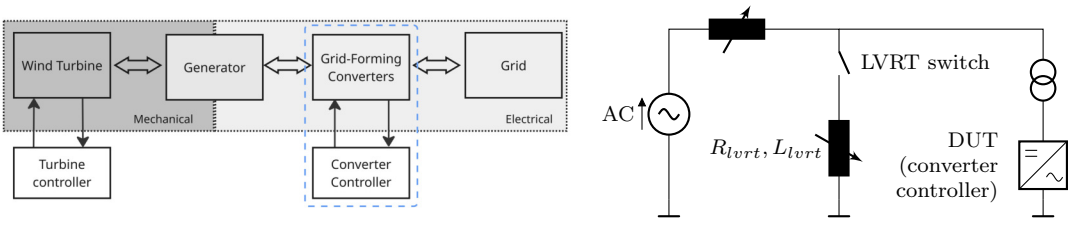


Figure 1: Conceptual architecture of the grid-forming wind turbine generator with converters (left) and an electrical diagram of the test-bed (right)

adaptive conformal prediction intervals using the variance of GP's output. Additionally, we extend the approach to divide the search space region into polytope-shaped sub-regions that are convex, for scalable and tighter bounds during sub-region type classification.

3. Converter Controller for a Grid-Forming Wind Turbine Generator (WTG)

A converter controller for a grid-forming wind turbine generator (WTG), is used as one of our target case studies. The primary role of a grid-forming WTG is to actively regulate an electrical grid through a converter-based control system. The overall concept is illustrated in the diagram shown on the left in Fig. 1, where the wind turbine and its controller resides within the mechanical domain whereas the grid-forming converters, associated controllers and the grid operate within the electrical domain of the system. Depending on the operating environment and the power usage from the consumer side, several scenarios can arise including deviation from the nominal voltage frequency, imbalance in the system voltage, and harmonic distortions in both voltage and current. In this work, we focus on testing the low voltage ride through (LVRT) capability of the converter controller, which maintains the voltage level such that the grid stays connected to the generator in the event of a voltage dip in the grid. A circuit diagram of our test-bed is shown on the right in Figure 1 where the size of voltage dip is controlled via simulation parameters R_{lvrt} and L_{lvrt}. The illustrated WTG setup is from the work in (DNV AS, 2022) where a set of predefined simulation scenarios and model parameters were used for testing the controller's LVRT capability.

According to the ISO 15026-1, assurance is defined as "grounds for justified confidence that a claim has been or will be achieved". Continuous assurance further extends this notion by emphasizing the ongoing generation of evidence and systematic testing integrated into development pipelines. This shift necessitates a testing framework that enables extensive exploration of test-cases, rather than relying on a fixed set of predefined scenarios. Therefore, in this work we further extend the previous test setup by incorporating a conformal prediction framework to provide statistically grounded confidence estimates. In particular, we investigate the controller's capability (depicted as a blue dotted box in Figure 1) for the scenarios where previous test results get invalidated due to activities such as software updates, physical asset replacements, changes in the operating environment, or gradual system's performance degradation due to aging. As such, we consider a continuous assurance process for validating claims in the form of contracts that are evaluated via monitoring and periodic assessments.

4. Backgrounds

4.1. Signal Temporal Logic

Signal Temporal Logic (STL) (Donzé and Maler, 2010) is a formal specification language that defines a system's behaviors with predicates over real-value signals as well as real-time constraints. As a result, STL is mostly used to evaluate time-series data obtained from the system under verification. In this work, we formulate *requirement contract* for the simulation model's behavior using STL specifications.

One notable feature of STL is its *quantitative* semantics, which enable measuring how well the system conforms to the given STL specification by computing robustness satisfaction rather than a binary (true or false) verdict. A set of STL kernel grammars is defined as below:

$$\varphi := \gamma \mid \neg\varphi \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 U_{[a,b]} \varphi_2$$

where γ is an atomic predicate. For a given signal trace $\xi = (t_1, \mathbf{x}_1), ..., (t_n, \mathbf{x}_n),\ \mathbf{x} \in \mathbb{R}^N$, which is a sequence of time-value pairs, a boolean satisfaction of an STL formula φ is defined as

$$\begin{aligned}
\xi &\models \gamma && \iff f(\mathbf{x}_0, t_0) > 0 \\
(\xi, t_k) &\models \gamma && \iff f(\mathbf{x}_k, t_k) > 0 \\
(\xi, t_k) &\models \varphi_1 \vee \varphi_2 && \iff (\xi, t_k) \models \varphi_1 \text{ or } (\xi, t_k) \models \varphi_2 \\
(\xi, t_k) &\models \neg\varphi && \iff \neg((\xi, t_k) \models \varphi) \\
(\xi, t_k) &\models \varphi_1 U_{[a,b]} \varphi_2 && \iff \exists t' \in [t_k + a, t_k + b] \text{ s.t.} \\
& && \quad (\xi, t') \models \varphi_2 \wedge \forall t'' \in [t_k, t'], (\xi, t'') \in \varphi_1
\end{aligned}$$

and a set of derived operators as:

$$\begin{aligned}
(\xi, t_k) &\models \varphi_1 \wedge \varphi_2 && \iff \neg(\neg\varphi_1 \vee \neg\varphi_2,) \\
(\xi, t_k) &\models \varphi_1 \rightarrow \varphi_2 && \iff \neg\varphi_1 \vee \varphi_2 \\
(\xi, t_k) &\models \Diamond_{[a,b]} \varphi && \iff \top U_{[t_k+a, t_k+b]} \varphi \\
(\xi, t_k) &\models \Box_{[a,b]} \varphi && \iff \neg\Diamond_{[t_k+a, t_k+b]} (\neg\varphi)
\end{aligned}$$

Quantitative semantics of STL for computing *robustness metric* is defined below using a function $\rho^\varphi : 2^S \times \mathbb{R} \rightarrow \mathbb{R}$ where S is a set of signal traces.

$$\begin{aligned}
\rho^\gamma(\xi, t_k) &= f(\mathbf{x}_k, t_k) \\
\rho^{\neg\varphi}(\xi, t_k) &= -\rho^\varphi(\xi, t_k) \\
\rho^{\varphi_1 \vee \varphi_2}(\xi, t_k) &= \max(\rho^{\varphi_1}(\xi, t_k), \rho^{\varphi_2}(\xi, t_k)) \\
\rho^{\varphi_1 U_{[a,b]} \varphi_2}(\xi, t_k) &= \max_{t' \in [t_k+a, t_k+b]} \Big(\min(\rho^{\varphi_2}(\xi, t'), \min_{t'' \in [t_k, t']} (\rho^{\varphi_1}(\xi, t'')) \Big)
\end{aligned}$$

In this paper, we consider a basic arithmetic operation as the custom metric function f, which returns a real value at a time t_k. In case f is in the form of inequality, we move all terms to one side and omit the inequality operator, for example $x < c \iff x - c$.

4.2. Gaussian Process Regression

Gaussian process (GP) regression is a probabilistic, non-parametric method used to construct a surrogate model that approximates the underlying distribution, which is assumed to be gaussian. Suppose a robustness metric ρ^{φ} defined by a user using STL. A GP surrogate model is then trained on samples $(\mathbf{x}_i, y_i)_{i=0}^{n}$ where $\mathbf{x}_i \in \mathbb{R}^N,\ y_i = \rho^{\varphi}(\xi_i, t_k)$ for N-d input space. The function output y_i is the result of the STL robustness metric and is assumed to be gaussian: $y_i \sim \mathcal{N}(\mu, \Sigma)$, where μ is the mean vector and Σ is $n \times n$ covariance matrix that is symmetric positive definite.

Given a kernel function $k(\mathbf{x}_i, \mathbf{x}_j)$, which describes similarity between two samples $\mathbf{x}_i, \mathbf{x}_j$ via their robustness values y_i, y_j, the first step of GP regression is to maximize the hyperparameters of the function that best fits the sampled data. In this work, we use RBF (Radial Basis Function) as a kernel function and log-marginal likelihood as an objective function for the kernel hyperparameter tuning.

For posterior prediction of y^* with an input $\mathbf{x}^*$, a joint (multivariate) distribution is also a Gaussian:

$$\begin{bmatrix} y \\ y^* \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu_y \\ \mu_{y^*} \end{bmatrix}, \begin{bmatrix} \Sigma_{\mathbf{xx}} + \epsilon, & \Sigma_{\mathbf{xx}^*} \\ \Sigma_{\mathbf{x}^*\mathbf{x}}, & \Sigma_{\mathbf{x}^*\mathbf{x}^*} \end{bmatrix} \right) \tag{1}$$

where ϵ is an observation noise and $\Sigma_{\mathbf{xx}'} = k(\mathbf{x}, \mathbf{x}')$. Then, the *conditioning* on the observed data y gives the posterior distribution for y^*:

$$y^* \mid y \sim \mathcal{N}\left(\mu_y + \Sigma_{\mathbf{xx}^*}(\Sigma_{\mathbf{x}^*\mathbf{x}^*} + \epsilon)^{-1}(y - \mu_y),\ \Sigma_{\mathbf{xx}} - \Sigma_{\mathbf{xx}^*}(\Sigma_{\mathbf{x}^*\mathbf{x}^*} + \epsilon)^{-1}\Sigma_{\mathbf{x}^*\mathbf{x}}\right) \tag{2}$$

In this work, we perform GP regression over a set of bounded regions to construct surrogate models and estimate each model's robustness with respect to the contract specification in its corresponding region. The decision of whether each region conforms to the contract is made using the conformal prediction framework inspired by (Qin et al., 2024).

4.3. Conformal Prediction

The core idea of conformal prediction (Vovk et al., 2005; Lei and Wasserman, 2014) is to construct an interval that, under the assumption of *exchangeability* of samples, provide a marginal coverage guarantee – meaning the interval is expected to contain the next sample from an arbitrary distribution with a specified probability. Given a sequence of data points $(\mathbf{x}_i, y_i)_{i=1}^{n}$ sampled from a joint distribution $\mathcal{D}_{\mathbf{x}y}$, the conformal prediction framework aims to construct a prediction set $C(\mathbf{x}_{n+1}) \subseteq \mathbb{R}$ for the next sample $(\mathbf{x}_{n+1}, y_{n+1})$ such that

$$\mathbb{P}(y_{n+1} \in C(\mathbf{x}_{n+1})) \geq 1 - \alpha$$

where $\alpha \in [0, 1]$ is a miscoverage level. Estimating inclusion of new data point in the prediction set involves computing $(1-\alpha)$-th quantile of the *conformity scores* $R_i = |y_i - \hat{\mu}(\mathbf{x}_i)|,\ i = 0, ..., n+1$ so that

$$C_{1-\alpha}(\mathbf{x}_{n+1}) = \left\{ y \in \mathbb{R} : \left(1 + \sum_{i=1}^{n} \mathbb{1}\{R_i \leq R_{n+1}\} \right) \leq \lceil (1-\alpha)(n+1) \rceil \right\} \tag{3}$$

where $\hat{\mu}$ is an estimator of the target regression function for the underlying distribution and $\mathbb{1}$ is an indicator function. To make any prediction if $y_{n+1} \in C(\mathbf{x}_{n+1})$, however, requires performing n regressions on the training data points as well as conformity score comparisons.

To avoid such extra overhead, Papadopoulus et. al. (Papadopoulos et al., 2002) has introduced an alternative approach called *split conformal prediction* that estimates $C(\mathbf{x}_{n+1})$ more efficiently by splitting the dataset into two disjoint subsets $(\mathbf{x}_i, y_i),\ i \in \mathcal{I}_1, \mathcal{I}_2,\ |\mathcal{I}_1| + |\mathcal{I}_2| = n$. The main idea of split conformal prediction is to construct the estimator $\hat{\mu}$ and conformity scores R_i separately from $\mathcal{I}_1$ and $\mathcal{I}_2$, respectively. As a result, the prediction set for the next $(\mathbf{x}_{n+1}, y_{n+1})$ can be computed via

$$C_{1-\alpha}(\mathbf{x}_{n+1}) = [\hat{\mu}(\mathbf{x}_{n+1}) - \hat{q}_{\mathcal{I}_2},\ \hat{\mu}(\mathbf{x}_{n+1}) + \hat{q}_{\mathcal{I}_2}] \tag{4}$$

where $\hat{q}_{\mathcal{I}_2}$ is $\lceil(|\mathcal{I}_2|+1)(1-\alpha)\rceil$-th smallest value in $\{R_i : i \in \mathcal{I}_2\}$.

The term $\hat{q}_{\mathcal{I}_2}$ in Eq. 4 is a constant residual derived R_i, whose accuracy may degrade when the variability of this residual is significant across $\mathbf{x}$. To accommodate this variability in the prediction, (El Mekkaoui et al., 2023; Papadopoulos et al., 2008) shown the use of either the mean or the standard deviation to adaptively adjust the prediction set based on the variance of $\mathbf{x}$. In this case, a new prediction set is given by

$$C_{1-\alpha}(\mathbf{x}_{n+1}) = [\hat{\mu}(\mathbf{x}_{n+1}) - \hat{\sigma}_{n+1}\hat{q}_{\mathcal{I}_2},\ \hat{\mu}(\mathbf{x}_{n+1}) + \hat{\sigma}_{n+1}\hat{q}_{\mathcal{I}_2}] \tag{5}$$

where $\hat{q}_{\mathcal{I}_2}$ is $\lceil(|\mathcal{I}_2|+1)(1-\alpha)\rceil$-th smallest value in $\{R'_i : i \in \mathcal{I}_2\}$ and R'_i is

$$R'_i = \left\{ y_i \in \mathcal{I}_2 : \frac{|y_i - \hat{\mu}(x_i)|}{\hat{\sigma}_i} \right\}$$

4.4. Linearly Constrained Regions

In this work, we partition and classify the parameter space of a simulation model, which we define as a *region*.

Definition 1 (Region) *Region is a n-dimensional hyper-rectangle bounded by $x \in \mathbb{R}^{n\times 2}$ limits.*

We aim to partition a region into smaller sub-regions, which also may take the form of hyper-rectangle or a set of convex polytopes defined by a finite number of half-spaces. In this case, we call such a region *linearly constrained region*.

Definition 2 (Linearly Constrained Region) *Linearly constrained region is a region with additional bounds defined by a set of linear inequalities in the form of $Ax \leq b$, where $A \in \mathbb{R}^{m\times n}$ and $b \in \mathbb{R}^m$.*

A region of simulation parameters, when partitioned using linear constraints, results in a set of convex polytopes. Each simulation, configured with sampled parameters, is evaluated against a contract defined as STL formula (Section 4.1). The resulting robustness metric serves as the basis for region classification. Note that in this paper, we use two interchangeable terms, each, when referring to samples from the input search space x (parameters or sample points) and samples from the output space y (STL robustness metric or sampled values). Using conformal prediction method outlined in Section 4.3, we classify each region into one of the following classes:

1. *Safe* – When the conformal prediction interval (Section 4.3) shows the region has a marginal coverage guarantee with $1-\alpha$ such that when next x is sampled in this region, it's STL robustness metric will be greater than zero.
2. *Unsafe* – Similar to safe region, but the guarantee is that the robustness metric will be less than zero.
3. *Undecided* – When the prediction interval contains both safe and unsafe boundary, i.e. when the interval contains zero, the corresponding region is classed as "unknown".

5. Classifying Linearly Constrained Regions using Conformal Prediction

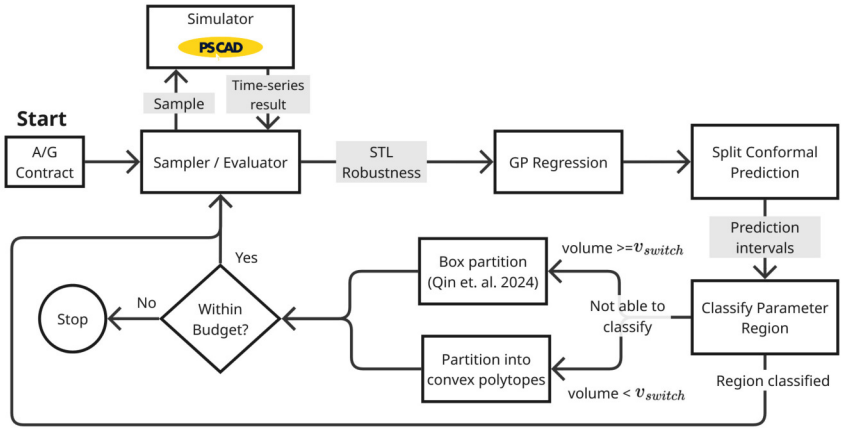


Figure 2: A graphical workflow for exploring a model's parameter space using the conformal prediction framework

The overall workflow of the conformal prediction framework for classifying parameter regions used for testing the grid-forming wind turbine converter is shown in Figure 2. As an initial step, a user provides a requirement contract specified in STL as an input to the workflow. In addition, the framework gets a set of parameters, including search space ranges, sampling size etc., which are used to configure simulations of the grid-forming controller in PSCAD[1], a modeling tool for electrical power systems.

5.1. Sampling Data from Simulation

A sampler triggers simulation in PSCAD with generated parameter values and receives time-series data as a result, which is evaluated against the requirement contract. Parameters of a simulation model and the STL robustness value are used as inputs and outputs of the training data for GP regression. A trained GP is the basis for the conformal prediction framework to classify, with statistical guarantee, if the next parameter to be drawn from the region would conform to the contract with a user-provided confidence level $1-\alpha$. Finding the parameter values that give the (estimated) maximum and minimum STL robustness

1. https://www.pscad.com

Algorithm 1: `SampleFromSimulator`

Input : $R = \langle \mathcal{T}_R, \mathcal{X}_R, A, b, - \rangle$: a search region, φ: requirement contract, n_{min}: minimum number of samples for a region, n_{budget}: available budget before sampling

Output: n'_{buget}: available budget after sampling, X_R, Y_R: sets of sampled points and values in R

1 $(\bar{X}_x, \bar{Y}_y) \leftarrow$ `GetCachedSamples`$(A,\ b,\ \mathcal{X}_R)$
2 **if** $|\bar{X}_x| < n_{min}$ **then**
3 **if** $n_{budget} - (n_{min} - |\bar{X}_x|) > 0$ **then**
4 $X_R \leftarrow$ `UniformSampleInRegion`$(A,\ b,\ \mathcal{X}_R,\ n_{min} - |\bar{X}_x|)$
5 **else**
6 $\mathcal{T}_R \leftarrow \perp$
7 **return** $n_{budget},\ \emptyset,\ \emptyset$
8 **end**
9 **else**
10 $X_R \leftarrow \bar{X}_x$
11 **end**
12 $Y_R \leftarrow$ `RunSimulator`$(X_R,\ \varphi)$
13 $n'_{budget} \leftarrow n_{budget} - |X_R|$
14 **return** $n'_{budget},\ X_R \cup \bar{X}_x,\ Y_R \cup \bar{Y}_y$

values, from GP combined with conformal prediction intervals, provides a basis to classify whether the region is safe or unsafe (i.e. satisfies the contract).

If the region cannot be classified with the given confidence level, we further partition the region into sub-regions and continue classification until 1) all of the partitioned regions are classified, or 2) the volume of the region becomes smaller than a threshold parameter, or 3) the algorithm has reached the maximum computation budget limit. These methods are adopted from (Qin et al., 2024; Pedrielli et al., 2023) but we extend their approach with a capability to partition a region not only into hyper-rectangles, but into convex polytopes. This way, the partitioned regions can fit better at the region class boundaries which otherwise would be classified "undecided" in the original approach, i.e. the prediction interval in the region contains both satisfaction and violation robustness values.

Algorithm 1 shows a sampling and evaluation of simulation results from PSCAD. A parameter search region R is defined as a region class $\mathcal{T}_R \in [0, 1, \perp]$ that denotes one of unsafe (0), safe (1), and undecided ($\perp$) types, region bounds $\mathcal{X}_R \in \mathbb{R}^{n \times 2}$, and a set of linear constraints defined by A and b that form linear boundaries and make the region polytope. In case the region shape is hyperrectangle in n-dimension, A and b are empty. Lastly, the operator '$-$' computes a new set of polytopes via the set difference between two regions. The function GetCachedSamples returns one or more sampled points and their corresponding values from the *prior* simulation runs. These points are drawn from the search space R and satisfy the specified constraints. If the number of cached samples is less than the required sample size, the difference is filled using a uniform sampling method, as illustrated in line 4, provided that the additional samples do not exceed the total budget limit n_{total}. In case this computation budget exhausts before classification concludes, the current region is classified "undecided" and the updated region is returned (line 7). The input data for a PSCAD simulator is configured and the result is obtained from RunSimulator function. This function also computes the STL robustness metric(s) (sample values) Y_R. Each contract φ

Algorithm 2: `ClassifyRegion`

Input : $R = \langle \mathcal{T}_R, \mathcal{X}_R, A, b, - \rangle$: a search region, α: miscoverage level, X_R, Y_R: sampled points and values in R
Output: k: classified region type

1 $(\hat{\mu},\ \hat{q}) \leftarrow$ `ConformalPrediction`$(X_R,\ Y_R,\ \alpha)$
2 $\hat{y}_{max} \leftarrow \max_{x_i \in R}\ \hat{\mu}(x_i) + \hat{\sigma}_i \hat{q} \quad$ s.t. $Ax_i \leq b$
3 $\hat{y}_{min} \leftarrow \min_{x_i \in R}\ \hat{\mu}(x_i) - \hat{\sigma}_i \hat{q} \quad$ s.t. $Ax_i \leq b$
4 **if** $\hat{y}_{min} \geq 0$ **then**
5 | **return** 1
6 **else if** $\hat{y}_{max} \leq 0$ **then**
7 | **return** 0
8 **else**
9 | **return** $\perp$
10 **end**

encoded in STL formula is in the form below

$$\varphi \ \models\ \square_{[0,t_{end}]}(A \rightarrow G) \tag{6}$$

where A is a set of assumptions $\{A_1, .., A_n\}$ and G is the associated guarantee, given the assumptions hold, as defined in (Benveniste et al., 2018). We compute a robustness metric at $t = 0$, which gives a single scalar value that indicates how well a time-series signal trace generated from the simulation satisfies or violates the contract. The remainder of the algorithm updates and returns the remaining computation budget, sampled points, and corresponding sampled robustness values.

5.2. Classifying a Region

After each cycle of sampling, simulating and evaluating points, a regression model is trained and used as a predictor for the next sample point. This prediction serves as the basis for classifying the current region being evaluated. In Algorithm 2, this is illustrated where GP is used as a regression model for prediction. In line 1, the split conformal prediction gives a GP model $\hat{\mu}$ and an estimated quantile $\hat{q}$ for the next predicted value from this model. We use the approach as in (El Mekkaoui et al., 2023) where $\hat{q}$ is computed by adjusting residual value of the conformity scores (Eq. 5) using the variance given by $\hat{\mu}$. The miscoverage level α is used by the algorithm to train the model that has $1 - \alpha$ confidence level for the next predicted value. The remainder of the algorithm checks if the estimated upper and lower bounds of $\hat{\mu}$ adjusted with the prediction interval $\hat{\sigma}_i \hat{q}$, where $\hat{\sigma}_i$ is obtained from the GP prediction, encompasses the classification boundary defined by an STL robustness value of zero. Linear constraints $Ax_i \leq b$ provided to the optimization algorithm bound the feasible solution space within the defined polytope region. In this work, we use GP Expected Improvement as an acquisition function for optimization shown in lines 2 and 3.

5.3. Partitioning a Region

If the region is classified "undecided" ($\perp$) by `ClassifyRegion`, the region is further divided into smaller sub-regions by Algorithm 3. The algorithm first checks if the region contains

Algorithm 3: `Partition`

Input : $R = \langle \mathcal{T}_R, \mathcal{X}_R, A, b, - \rangle$: a search region to be partitioned, X_R, Y_R: sampled points in R, *method*: region partitioning method, v_{min}: minimum volume threshold, n_{pmax}: maximum depth of the partitioned polytopes

Output: $\mathcal{R}$: a set of partitioned regions

1 **if** $\exists y, y' \in Y_R$ *s.t.* $y > 0 \ \wedge \ y' < 0 \wedge method \neq box$ **then**
2 **if** *ploytope* = *subtraction* **then**
3 $X^- \leftarrow \{x_i \in X_R \ : \ y_i < 0\}$
4 $R^- \leftarrow$ `GenerateRegionFromPoints`(X^-)
5 $\mathcal{R} \leftarrow R - R^-$
6 **if** $|\mathcal{R}| + 1 > n_{pmax}$ **then**
7 **return** `BoxPartition`(R)
8 **else if** $\frac{\sum_{r \in \mathcal{R}} \texttt{Vol}(r)}{\texttt{Vol}(R)} < v_{min}$ **then**
9 **return** `BoxPartition`(R)
10 **else**
11 **return** $\mathcal{R} \cup \{R^-\}$
12 **else if** *method* = *clustering* **then**
13 **return** `ClusteringPartition`(R, X_R, Y_R, α)
14 **else**
15 **return** `BoxPartition`(R)
16 **end**

both sample values that violate or satisfy the contract so that the region can be partitioned into "safe" or "unsafe" sub-regions. If partitioning is feasible, the region is partitioned using one of the following methods based on the partitioning *method* is chosen:

1. **Partitioning at a vertex (Box partition)** – This partitioning method is from (Qin et al., 2024; Pedrielli et al., 2023) where the region is split into $2^{|n|}$ sub-regions at a shared vertex aligned along the parameter axis and bounded by χ_R. This is done by the function BoxPartition at line 15. The vertex is chosen via Bayesian optimization using a custom UCB (upper confidence bound) acquisition function shown below that optimizes the region towards where zero crossing occurs, i.e. a boundary between safe and unsafe regions.

$$\text{maximize} \ -(|\mu| + \beta\sigma) - \rho \quad \text{s.t. } \rho = \sum_{x \in \chi_R} (\epsilon - dist_{min}(x, \chi_R)) \cdot w_{pen} \cdot c \tag{7}$$

$$c = \begin{cases} 1 & \text{if } dist_{min}(x, \chi_R) < \epsilon \\ 0 & \text{otherwise} \end{cases}$$

In addition, we employ a penalty term ρ if the solution is too close to the region boundary χ_R. $dist_{min}$ is a function that returns the closest distance of a point to the boundary χ_R, β is an exploration and exploitation parameter for UCB, w_{pen} is a penalty weight, and ϵ is the minium allowable distance to the region boundary before the penalty is applied. The crux of having this penalty term is to avoid the optimization trapped at the previously chosen vertexes and also to avoid creating too many so called sliver regions.

Algorithm 4: `ClusteringPartition`

Input : $R = \langle \mathcal{T}_R, \mathcal{X}_R, A, b, - \rangle$: a search region, α: miscoverage level, X_R, Y_R: sampled points
Output: $\mathcal{R}$: a set of partitioned regions

1 $Y_{signs} \leftarrow$ `Sign`(Y_R)
2 $(A_{lda}, b_{lda}) \leftarrow$ `LinearDiscriminantAnalysis`(X_R, Y_{signs})
3 **if** `LDAAccuracy`(`Sign`$(A_{lda} X_R + b_{lda})$, Y_{signs}) $> 1 - \alpha$ **then**
4 | **return** `DivideWithPlanes`$(R,\ A_{lda},\ b_{lda})$
5 $L \leftarrow$ `ClusterPoints`(X_R, Y_{signs})
6 $(C, f_{svc}) \leftarrow$ `LinearSVClassification`(X_R, L)
7 $\mathcal{R} \leftarrow \{R\}$
8 **for** $(A_c, b_c) \in C$ **do**
9 | $\mathcal{R}_{next} \leftarrow \emptyset$
10 | **for** $R_i \in \mathcal{R}$ **do**
11 | | $X_\perp \leftarrow$ `SampleHyperplane`$(A_c, b_c, \mathcal{X}_R)$
12 | | $X^+ \leftarrow X_\perp + 0.1 \cdot A_c, \quad X^- \leftarrow X_\perp - 0.1 \cdot A_c$
13 | | **if** $\exists i\ s.t.\ \mathbb{1}[f_{svc}(X_i^+) \wedge \neg f_{svc}(X_i^-)] = 1$ **then**
14 | | | $\mathcal{R}' \leftarrow$ `DivideRegionWithPlane`(R_i, A_c, b_c)
15 | | | $\mathcal{R}_{next} \leftarrow \mathcal{R}_{next} \cup \mathcal{R}'$
16 | | **else**
17 | | | $\mathcal{R}_{next} \leftarrow \mathcal{R}_{next} \cup \{R_i\}$
18 | **end**
19 | $\mathcal{R} \leftarrow \mathcal{R}_{next}$
20 **end**
21 **return** $\mathcal{R}$

2. **Partitioning with region subtraction** – A new region R^- is created using all the sample values in the current region violating the contract. We are using QuickHull and double description methods to create this region, as shown at lines 3 and 4. The new region is then subtracted from the original region R that encompasses it. The resultant region is further divided into a set of polytopes $\mathcal{R}$ by slicing the subtracted region with boundaries of R^-. If the number of generated polytopes + 1 is greater than n_{pmax} or the sum of their volumes is smaller than the relative volume of their parent region R defined by v_{min}, we re-partition R using BoxPartition.

3. **Partitioning with clustering** – This method aims to partition a region using a data clustering method in the augmented sample space. A pseudo-code of this approach called ClusteringPartition is shown in Algorithm 4.

Algorithm 4 first checks if the sample points can be easily separated, based on their signs, using a linear decision plane determined by Linear Discriminant Analysis (LDA) (lines 1-4). The accuracy of this separation is evaluated by LDAAccuracy that computes a ratio of the number of points classified by LDA to the ground truth Y_{signs}. If the accuracy level is grater than $1 - \alpha$, separation is considered successful and the regions divided by the linear plane are returned. Otherwise, the algorithm continues dividing the region using Linear Support Vector Classification (SVC), which creates a set of linear planes C along with the classifier f_{svc}. The sample points are labeled using their standard score and signs of their values (STL robustness).

Algorithm 5: The main loop for classifying regions using conformal prediction

Data: $\mathcal{R}_{safe}, \mathcal{R}_{unsafe}, \mathcal{R}_{undecided}$: sets of classified regions, $R = \langle \mathcal{T}_R, \mathcal{X}_R, A, b, - \rangle$: a root region, φ: requirement contract, α: miscoverage level, *method*: partitioning method, v_{switch}: threshold for switching to polytope partitioning, v_{min}: minimum volume threshold, n_{pmax}: maximum depth of polytopes, n_s: minimum number of samples, n_{total}: total budget limit,

1 $\mathcal{R} \leftarrow \{R\}$
2 $n_{budget} \leftarrow n_{total}$
3 **for** $R_i \in \mathcal{R}$ **do**
4 $\quad (n_{budget}, X_R, Y_R) = \texttt{SampleFromSimulator}(R_i, \varphi, n_s, n_{budget}, n_{total})$
5 $\quad$ **if** $X_R = \emptyset \vee Y_R = \emptyset$ **then**
6 $\quad\quad \mathcal{R}_{undecided} \leftarrow \mathcal{R}_{undecided} \cup \{R_i\}$
7 $\quad$ **else**
8 $\quad\quad k \leftarrow \texttt{ClassifyRegion}(R_i, \alpha, X_R, Y_R)$
9 $\quad\quad$ **if** $k = 1$ **then**
10 $\quad\quad\quad \mathcal{R}_{safe} \leftarrow \mathcal{R}_{safe} \cup \{R_i\}$
11 $\quad\quad$ **else if** $k = 0$ **then**
12 $\quad\quad\quad \mathcal{R}_{unsafe} \leftarrow \mathcal{R}_{unsafe} \cup \{R_i\}$
13 $\quad\quad$ **else if** $\frac{\texttt{Vol}(R_i)}{\texttt{Vol}(R)} \leq v_{min}$ **then**
14 $\quad\quad\quad \mathcal{R}_{undecided} \leftarrow \mathcal{R}_{undecided} \cup \{R_i\}$
15 $\quad\quad$ **else**
16 $\quad\quad\quad$ **if** $\frac{\texttt{Vol}(R_i)}{\texttt{Vol}(R)} \leq v_{switch}$ **then**
17 $\quad\quad\quad\quad method' \leftarrow method$
18 $\quad\quad\quad$ **else**
19 $\quad\quad\quad\quad method' \leftarrow$ "box"
20 $\quad\quad\quad \mathcal{R}_p \leftarrow \texttt{Partition}(R_i, X_R, Y_R, method', v_{min}, n_{pmax})$
21 $\quad\quad\quad \mathcal{R} \leftarrow \mathcal{R} \cup \mathcal{R}_p$
22 **end**

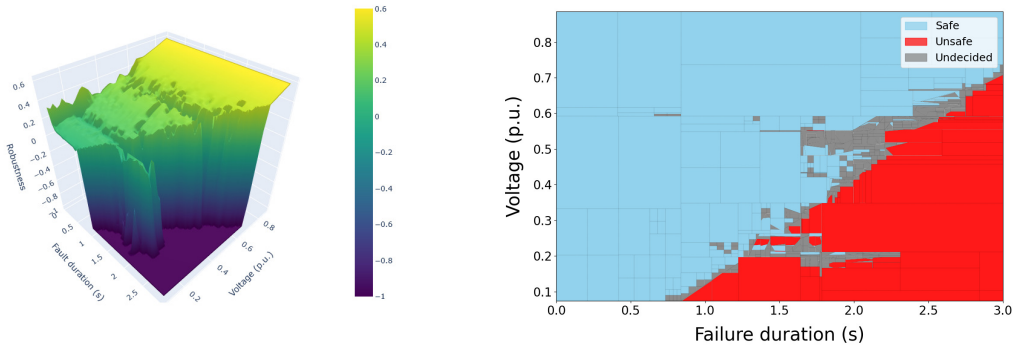
In the next step, SampleHyperPlane generates test samples that are projected onto each hyperplane created by SVC. By translating the samples into each of the class regions, but also close to the hyperplane boundary, we evaluate the quality of the linear decision plane by the SVC model. This is done using the function f_{svc} (lines 12-13), which returns true if the model correctly predicts the class of each test sample across the hyperplane boundary. The algorithm iterates and continues partitioning the sub-regions (polytopes) until the generated linear plane fails to find a boundary for separating samples. Finally, a list of partitioned sub-regions is returned as result by the algorithm.

The main loop of the region classification is shown in Algorithm 5, which employs the aforementioned sub-algorithms to classify partitioned search space regions. Candidate samples are first drawn from the root region R, which is used to classify the region using the conformal prediction method in ClassifyRegion. As long as a region cannot be classified as either safe or unsafe (i.e. $k = \perp$), it is further partitioned by Partition with the partitioning method defined by *method*. Note that in this algorithm, we passed the observed samples X_R, Y_R for partitioning the region. Alternatively, sample points may be drawn from uniform random posterior of $\hat{\mu}$ with the means adjusted using the estimated quantiles $\hat{\sigma}_i \hat{q}$. The corresponding algorithm is provided in Appendix A.

The algorithm first uses "box" (BoxPartition) if the current region is sufficiently large, and switches to one of the polytope partitioning methods (clustering or subtraction) when the relative volume of the partitioning sub-region to the root R gets smaller than v_{switch}. The main loop terminates when all sub-regions have been classified or the total number of

Table 1: Experimental settings for simulations of the converter controller in the grid-forming wind turbine generator

Setting	Value	Setting	Value
Requirement contract	$\square_{[3,10]}(s_{switch} < 1.0 \rightarrow V_{rms} > 0 \wedge s_{trip} < 0)$	Polytope threshold (v_{switch})	7×10^{-3}
Voltage dip level (p.u.)	$[0.0737, 0.8855]$	Minimum volume (v_{min})	4×10^{-4}
Fault duration (s)	$[0.0, 3.0]$	UCB penalty weight (w_{pen})	0.5
Fault start time (s)	6.0	Simulation time (s)	10.0
Minimum sample size (n_{min})	40	Step size (μs)	2.0
Miscoverage level (α)	0.1		



(a) (b)

Figure 3: Experimental results for the converter controller of the grid forming wind-turbine generator – (a) Robustness surface showing the fault duration (y-axis) versus voltage dip in per unit (x-axis), and (b) partitioned region obtained using the subtraction method where the search space is per unit voltage drops versus failure duration.

samples reaches the user-defined budget limit n_{budget}. At this point, any remaining regions are labeled as "undecided" (see lines 5-6).

6. Experimental Results

In this section, we conduct a set of experiments to partition and classify parameter regions of benchmark simulation models, using the method described in the previous section.

6.1. A Controller for the Grid-Forming Wind Turbine Generator (WTG)

A converter controller in the grid-forming WTG is responsible for driving the converter that performs DC/AC conversion and maintains operation in case there are disturbances in providing voltage and current. In this experiment, we are testing if the converter controller maintains the voltage level and grid stays connected to the generator in the event of a voltage dip in the grid.

The configuration settings of the experiment is shown in Table 1. Requirement contract is defined to evaluate simulation between 3 and 10 seconds. This is due to the converter controller requiring at least 3 seconds for initialization before conducting tests. The contract

is checked until the end of simulation (10 seconds) such that if the LVRT switch gets closed, indicated by $s_{switch} < 1.0$, the controller maintains both the RMS voltage level above zero[2] $V_{rms} > 0$ and, in addition, the converter should not trip, which is monitored through the circuit breakers in the controller design and indicated by $s_{trip} < 0$. We aim to classify sub-regions to either safe or unsafe by partitioning until the ratio of sub-region's volume to the global region becomes less than the threshold v_{min}. Furthermore, we use the partition by polytope *subtraction* method when the relative volumes fall below v_{switch}.

The voltage dip is emulated using the instrumental switch that connects the generator line at the fault start time of 6 seconds. Here, the parameters to be explored are the resistance R_{lvrt} and inductance L_{lvrt} of the instrumental circuit. To compute the per unit (p.u.) value of the voltage dip with the black-box controller model, whose reactance is unknown, we first generated samples of R_{lvrt} and L_{lvrt} that sufficiently cover possible range of voltage dips. Prior to the experiment, a lookup table was created to map the p.u. voltage dip values to the corresponding R_{lvrt} and L_{lvrt} values using a linear interpolation. For a given p.u. voltage dip value, the corresponding R_{lvrt} and L_{lvrt} values are then estimated from this table. As a result, the search space is effectively reduced from three dimensions to two dimensions.

Figure 3(a) shows the surface plot of the STL robustness metric collected during the experiment. It was observed that, overall, the controller was able to keep the voltage level without tripping when the voltage and the duration of the switch being closed are kept within the theoretical ideal bound. However, it was observed that the controller frequently tripped unexpectedly at the time when the induced fault (voltage dip) was cleared. This behavior can be seen in Figure 3(a), where two horizontal valleys can be observed in the robustness surface along the fault-duration axis. As shown in the following experiment, such surface shape posed challenges for classifying regions in the vicinity of these areas.

The partitioned model parameter regions are shown in Figure 3(b), where the search space is defined by per unit voltage dip (p.u.) and the fault duration in seconds. The safe, unsafe and undecided regions are colored with light-blue, red, and gray, respectively. The overall shape of the safe and unsafe regions aligns with the general trend of the Low Voltage Ride Through capability of a WTG, see (European Commission, 2016), where the system tends to trip more frequently during larger voltage dips of longer duration. However, as shown in the robustness surface in Figure 3(a), irregular surface in the p.u. voltage range, particularly near 0.2 and between 0.5 and 0.65, resulted in a large area of undecided regions. Undecided regions can potentially be reduced by employing smaller v_{min} and v_{switch}. However, this will expectedly create also more, and smaller regions, which can significantly increase runtime of the classification process.

Due to the blackbox nature of the WTG controller, we were only able to estimate the per-unit voltage dip from the corresponding R_{lvrt} and L_{lvrt} values, which may not accurately reflect the ground-truth voltage dip and therefore limits its use as a baseline for comparison between different region partitioning methods. To provide further insight into the performance of the proposed method, we conducted additional experiments using

2. Note that although V_{rms} would drop to zero during the actual converter tripping event, the testbed was originally designed such that this was not the case. Instead, the experiment emulated tripping by mapping the voltage level to zero when the circuit breaker s_{trip} was opened

Table 2: Experimental settings of the mountain car benchmark

Setting	Value	Setting	Value
Requirement Contract	$\square_{[0,0]}(\top \rightarrow \Diamond_{[0,10]}(x \geq 0.45))$	Polytope threshold (v_{switch})	1×10^{-2}
Initial position (x)	$[-0.7, 0.2]$	Minimum volume ($v_{\min}$)	5×10^{-4}
Initial velocity (v)	$[-0.02, 0.02]$	UCB penalty weight (w_{pen})	0.35
Minimum sample size ($n_{\min}$)	80	Episode duration	10.0 (time unit)
Miscoverage level (α)	0.05		

the Mountain Car and F-16 control system benchmarks, where the input paramters can be directly controlled and the ground-truth STL robustness values can be obtained.

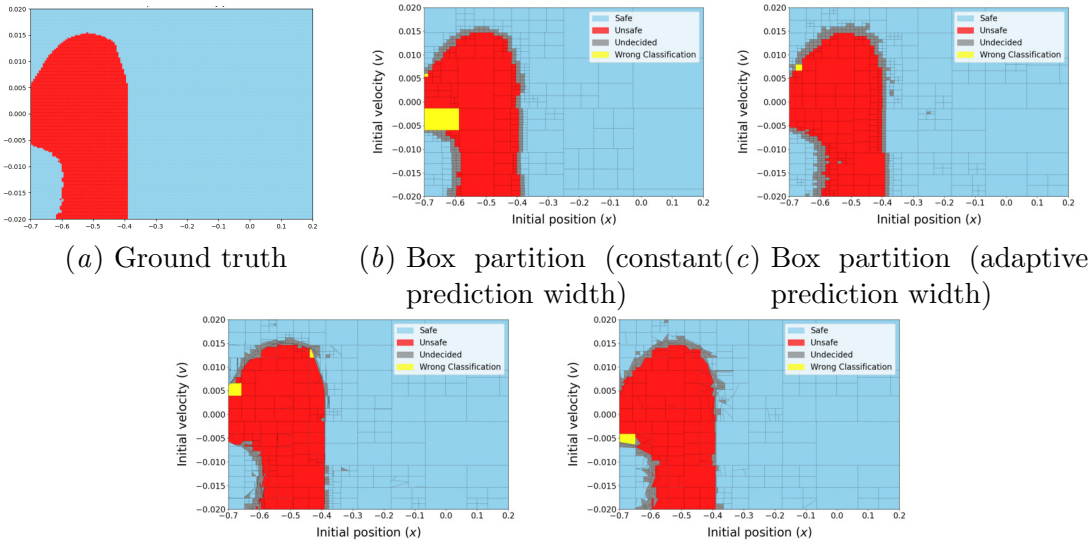
6.2. Mountain Car

The mountain car is a classic benchmark problem in reinforcement learning (RL) used to evaluate the performance of RL agents. Initially, the car is randomly placed within a sinusoidal valley with a given initial velocity. The agent's objective is to drive the car up to the top of the right hill by controlling its acceleration – either to the left or to the right. We used a Deep Q-Network (DQN) based RL agent[3] combined with the Gymnasium Python library[4]. An overview of the simulation parameter setting is shown in Table 2. Note that in the table, we included a $\square_{[0,0]}$ term in the requirement contract to maintain consistency with the notation presented in Eq. 6. However, this term can be omitted in this particular example. In addition, it is found that the DQN agent consistently succeeds in driving the car up the hill when given sufficiently long episode (simulation) duration. For this experiment, however, we restricted the episode length to 10 time unit.

Figure 4 presents visualizations of the partitioned regions alongside the ground truth. Similar to the WTG experiment, the safe, unsafe and undecided regions are colored with light-blue, red, and gray, respectively. Additionally, regions that are misclassified as safe (while ground truth is unsafe), or unsafe (while ground truth is safe) by the algorithm are highlighted in yellow. These misclassified regions are identified by checking whether any sample values from the ground truth are incorrectly placed in the regions produced by the partitioning method. Figures 4(c), 4(d), and 4(e) present the results for the methods described in Section 5, corresponding to box, polytope subtraction, and polytope clustering, respectively. Furthermore, we conducted one additional experiment using the box partitioning method with a constant prediction interval width, computed using Eq. 4, as shown in Figure 4(b). This experiment was performed to compare its performance against methods whose prediction intervals are adjusted based on the uncertainty estimated by the GP model. In the following, we abbreviate the these partitioning methods as BC (box partitioning with constant prediction width), BA (box partitioning with adaptive prediction width), PS (polytope subtraction), and PC (polytope clustering).

Quantitative comparisons of the different partitioning approaches are presented in Figure 5, which compares the total number of regions and the volumes of safe, unsafe, undecided, and misclassified regions across different miscoverage levels α. For the larger α of 0.3,

3. https://github.com/MehdiShahbazi/DQN-Mountain-Car-Gymnasium
4. https://github.com/Farama-Foundation/Gymnasium



(a) Ground truth (b) Box partition (constant prediction width) (c) Box partition (adaptive prediction width)

(d) Partition by subtraction (e) Partition by clustering

Figure 4: Results of box, subtraction and clustering-based partitioning methods for the mountain car

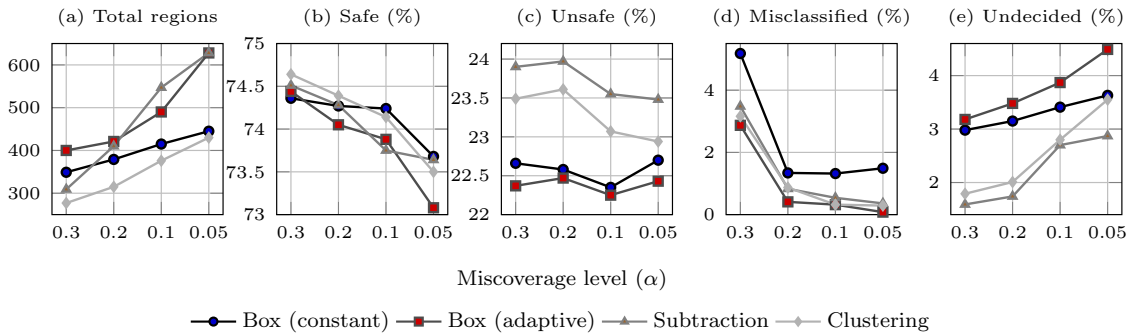


Figure 5: Trend in region count and proportion of classified regions (volumes) across miscoverage levels for the mountain car benchmark

PS and PC methods produced the fewer regions than both box partitioning methods. As the α decreased, however, the number of regions generated by BA and PS surpassed those of other approaches. This behavior was expected because the BA method generated more wider prediction intervals that cross class boundaries. For the PS case, it tends to create a larger number of sub-regions as a result of the subtraction algorithm, particularly when the class boundary is highly non-linear or difficult to estimate. This situation arises when true values close to zero are distributed more widely around the class boundary, resulting in increased region fragmentation. In average, both PS and PC methods produced larger safe and unsafe regions compared to the box partitioning methods across all α values, while BA method did not misclassify regions at $\alpha = 0.05$. The volumes of undecided regions

Table 3: Experimental settings of the F-16 GCAS benchmark

Experiment settings	Values	Experiment settings	Values
Requirement Contract	$\square_{[0,10]}(\top \rightarrow q \leq 37.0)$	Angle of attack (α_{aoa})	[-10.0, 0]
Pitch angle (θ)	[-30, 0]	Minimum sample size in each region (n_{min})	40
Miscoverage level (α)	0.1	Polytope partitioning threshold (v_{switch})	1×10^{-2}
Minimum volume threshold (v_{min})	5×10^{-4}	UCB penalty weight (w_{pen})	0.4
Initial altitude (ft)	1000	Initial velocity (ft/sec)	540
Initial roll angle (deg/s)	-22.5	Yaw angle (deg/s)	0
Side slip angle (deg/s)	0	Engine power level	9
Time step	1/30		

are increasing with smaller α values as expected, with the PS and PC methods producing smaller undecided regions than BA and BC.

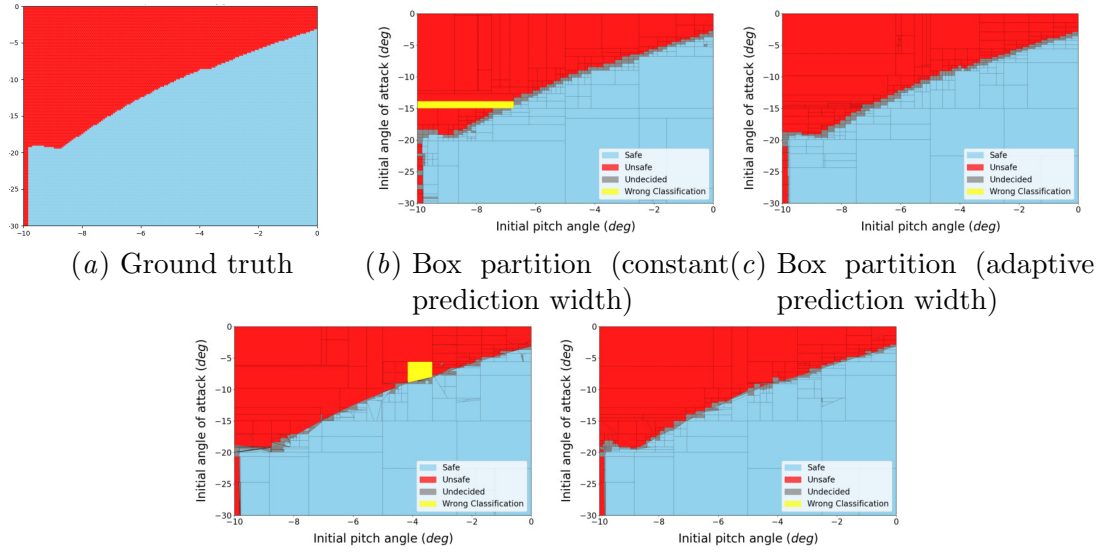
6.3. F-16 Control System (GCAS autopilot)

F-16 control system benchmark[5] from (Heidlauf et al., 2018) simulates an F-16 control system for verification and analysis purposes. We have chosen the GCAS (Ground Collision Avoidance System) autopilot scenario that detects and avoids ground collision scenario by switching its mode to "pull up". The experimental setting is shown in Table 3. Similar to the experiment conducted in (Qin et al., 2024), we vary the initial values of angle of attack α_{aoa} and pitch angle θ of the aircraft, and evaluate if pitch rate q remains within 37.0 deg/s. Results of the partitioning methods as well as the ground truth are shown in Figure 6. Unlike the mountain car benchmark, F-16 control system exhibits a smoother surface at the class boundaries.

Quantitative results are shown in Figure 7. Similar to the mountain car benchmark, the total number of regions created were increasing as α increased, while the polytope-based methods produced fewer regions than the box partitioning methods at $\alpha = 0.1$. Both polytope-based methods identified marginally larger safe and unsafe regions, except for the safe region classification at $\alpha = 0.2$. As α increased, the box partitioning methods tended to misclassify larger regions. Overall, the polytope-based methods resulted in smaller misclassified regions, in particular for larger α values. Yet, this advantage diminished as α decreased, as shown in Figure 7(d). This suggests that the polytope-based methods are more effective at classifying regions near the class boundaries. Nevertheless, a more accurate estimation of the uncertainty margin along with the class boundaries could improve region classification accuracy, for example, allocating more computational resources for sampling as well as min/max estimation on these uncertain regions. We leave this to future work.

The end-to-end runtimes for mountain car and GCAS benchmarks are shown in Figure 8. Overall, the polytope-based methods required longer runtimes. The runtimes of the partitioning algorithms depend largely on the simulation complexity and the optimization strategy used to estimate the GP-based minimum and maximum values within each region. For example, a single run of the WTG benchmark experiment took approximately 10-15 minutes, and the number of parallel simulations was limited to eight due to licensing constraints. Nevertheless, the sampling and partitioning tasks associated with disjoint regions

5. https://github.com/stanleybak/AeroBenchVVPython



(a) Ground truth (b) Box partition (constant prediction width) (c) Box partition (adaptive prediction width)

(d) Partition by subtraction (e) Partition by clustering

Figure 6: Results of box, subtraction and clustering-based partitioning methods for the F-16 GCAS benchmark

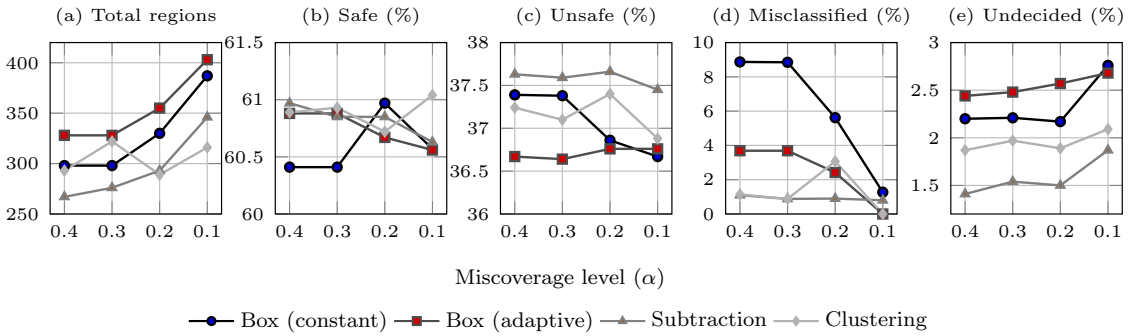
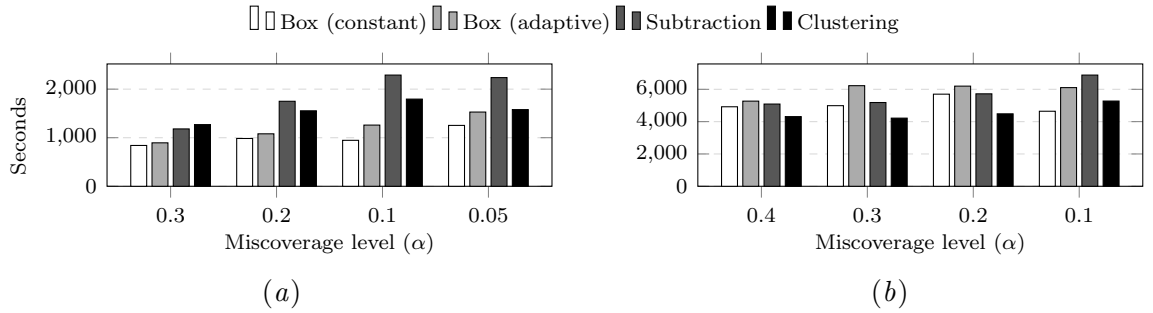


Figure 7: Trend in region count and proportion of classified regions (volumes) across miscoverage levels for the GCAS benchmark

are independent and can therefore be fully parallelized, potentially improving performance when supported by the simulation environment. The choice of optimization algorithm and its configuration (e.g. the number of initial samples and restarts) depends on the complexity of the underlying function being optimized. We note that finding the optimal settings for each benchmark is a non-trivial task and beyond the scope of this work.

7. Conclusion and Future Work

In this work, we explored the testing aspects of realizing a continuous assurance cycle by employing the conformal prediction framework for classifying satisfaction or violation of



(a) (b)

Figure 8: End-to-end runtime comparison on (a) mountain car and (b) GCAS benchmarks across different miscoverage levels (no parallelization)

regions according to a requirement contract defined in temporal logic. The basic idea of region partitioning and classification from (Qin et al., 2024) was extended with the ability to create linearly constrained regions forming convex polytopes. Eventually, the approach was evaluated to a set of benchmark cases including the converter controller for the grid-forming WTG. The experimental results showed that the polytope-based partitioning methods reduced the overall undecided region volume, however, there is a room for improvements to reduce the misclassified regions.

As a future work, we would like to investigate other methods such as Convex Polytope Machine (Kantchelian et al., 2014) or Convex Polytope Trees (Armandpour et al., 2021) for clustering and partitioning regions.

Acknowledgments

The authors would like to thank Ravi Singh and Marcel Eijgelaar for their support in providing access to the converter controller model and for assisting with the setup of the simulation environment used in the experiments. The authors also would like to thank Sara El Mekkaoui for providing valuable feedback.

This research is part of the FlexH2 project, which is supported with a subsidy by the Dutch Ministry of Energy and Climate. (MOOI: Missiegedreven Onderzoek Ontwikkeling en Innovatie, grant agreement 52103.) https://grow-flexh2.nl.

References

Mohammadreza Armandpour, Ali Sadeghian, and Mingyuan Zhou. Convex polytope trees. *Advances in Neural Information Processing Systems*, 34:5038–5051, 2021.

Erfan Asaadi, Ewen Denney, Jonathan Menzies, Ganesh J Pai, and Dimo Petroff. Dynamic assurance cases: a pathway to trusted autonomy. *Computer*, 53(12):35–46, 2020.

Albert Benveniste, Benoît Caillaud, Dejan Nickovic, Roberto Passerone, Jean-Baptiste Raclet, Philipp Reinkemeier, Alberto Sangiovanni-Vincentelli, Werner Damm, Thomas A Henzinger, Kim G Larsen, et al. Contracts for system design. *Foundations and Trends® in Electronic Design Automation*, 12(2-3):124–400, 2018.

HOPEWIND Technology Co. Ltd DNV AS, Ming Yang Smart Energy Group. Testing of grid-forming wind turbine generator system using controller hil tests. Technical report, DNV AS, Ming Yang Smart Energy Group, HOPEWIND Technology Co. Ltd, 2022.

Alexandre Donzé and Oded Maler. Robust satisfaction of temporal logic over real-valued signals. In *International conference on formal modeling and analysis of timed systems*, pages 92–106. Springer, 2010.

Sara El Mekkaoui, Carla J Ferreira, Juan Camilo Guevara G'omez, Christian Agrell, Nicholas James Vaughan, and Hans Olav Heggen. Neural networks based conformal prediction for pipeline structural response. In *Conformal and Probabilistic Prediction with Applications*, pages 134–146. PMLR, 2023.

European Commission. Commission regulation (eu) 2016/631 of 14 april 2016 establishing a network code on requirements for grid connection of generators. Official Journal of the European Union, L 112, pp. 1–68, April 2016. URL https://eur-lex.europa.eu/eli/reg/2016/631/oj. Accessed 2026-08-03.

Georg Hake, Carl Philipp Hohl, and Axel Hahn. Continuous contract based verification of updates in maritime shipboard equipment. *Journal of Marine Science and Engineering*, 9(7):688, 2021.

Peter Heidlauf, Alexander Collins, Michael Bolender, and Stanley Bak. Verification challenges in f-16 ground collision avoidance and other automated maneuvers. *ARCH@ ADHS*, 2018, 2018.

Alex Kantchelian, Michael C Tschantz, Ling Huang, Peter L Bartlett, Anthony D Joseph, and J Doug Tygar. Large-margin convex polytope machine. *Advances in Neural Information Processing Systems*, 27, 2014.

Jing Lei and Larry Wasserman. Distribution-free prediction bands for non-parametric regression. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 76(1): 71–96, 2014.

NASA. Reducing risk in software-driven hazards across nasa programs, April 2026. URL https://sma.nasa.gov/news/articles/newsitem/2026/04/08/reducing-risk-in-software-driven-hazards-across-nasa-programs-how-software-assurance-and-iv-v-strengthens-mission-safety.

Harris Papadopoulos, Kostas Proedrou, Volodya Vovk, and Alex Gammerman. Inductive confidence machines for regression. In *European conference on machine learning*, pages 345–356. Springer, 2002.

Harris Papadopoulos, Alex Gammerman, and Volodya Vovk. Normalized nonconformity measures for regression conformal prediction. In *Proceedings of the IASTED International Conference on Artificial Intelligence and Applications (AIA 2008)*, pages 64–69, 2008.

Giulia Pedrielli, Tanmay Khandait, Yumeng Cao, Quinn Thibeault, Hao Huang, Mauricio Castillo-Effen, and Georgios Fainekos. Part-x: A family of stochastic algorithms for

search-based test generation with probabilistic guarantees. *IEEE transactions on automation science and engineering*, 21(3):4504–4525, 2023.

Xin Qin, Yuan Xia, Aditya Zutshi, Chuchu Fan, and Jyotirmoy V Deshmukh. Statistical verification using surrogate models and conformal inference and a comparison with risk-aware verification. *ACM Transactions on Cyber-Physical Systems*, 8(2):1–25, 2024.

Philipp Schleiss, Francesco Carella, and Iwo Kurzidem. Towards continuous safety assurance for autonomous systems. In *2022 6th International Conference on System Reliability and Safety (ICSRS)*, pages 457–462. IEEE, 2022.

Tobias Rye Torben, Jon Arne Glomsrud, Tom Arne Pedersen, Ingrid B Utne, and Asgeir J Sørensen. Automatic simulation-based testing of autonomous ships using gaussian processes and temporal logic. *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, 237(2):293–313, 2023.

Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.

Fredrik Warg, Hans Blom, Jonas Borg, and Rolf Johansson. Continuous deployment for dependable systems with continuous assurance cases. In *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, pages 318–325. IEEE, 2019.

Ran Wei, Simon Foster, Haitao Mei, Fang Yan, Ruizhe Yang, Ibrahim Habli, Colin O'Halloran, Nick Tudor, Tim Kelly, and Yakoub Nemouchi. Access: Assurance case centric engineering of safety–critical systems. *Journal of Systems and Software*, 213: 112034, 2024.

Zhenya Zhang and Paolo Arcaini. Gaussian process-based confidence estimation for hybrid system falsification. In *International Symposium on Formal Methods*, pages 330–348. Springer, 2021.

Appendix A. Uniform Sample Posterior for Partitioning a Region

Y^u and Y^l are the upper and lower quantile posterior estimates, respectively. In this algorithm, our aim is to identity a set of sample points Y_R that contains values both below zero (indicating STL violation) and above zero (indicating STL satisfaction), enabling the region to be partitioned into two different classes.

If all upper quantile samples are strictly greater than zero (line 4), the algorithm returns the lower quantile samples as Y_R (line 5). On the other hand, if the upper quantile samples include values less than zero, the upper quantile samples are returned instead (line 7). The returned sample points are subsequently passed to Algorithm 3.

Algorithm 6: Uniform Sample Posterior from $\hat{\mu}$

Input : $R = \langle \mathcal{T}_R, \mathcal{X}_R, A, b, - \rangle$: a search region to be partitioned, $\hat{q}$: conformity score, $\hat{\mu}$: Gaussian Process model produced in ClassifyRegion, n_{min}: minimum number of sample

Output: X_R, Y_R: sample points in R

$X_R \leftarrow$ `UniformSampleInRegion`$(A,\ b,\ \mathcal{X}_R,\ n_{min})$

$Y^u \leftarrow \{\hat{\mu}(x_i) + \hat{\sigma}_i \hat{q} \mid x_i \in X_R, Ax_i \leq b\}$

$Y^l \leftarrow \{\hat{\mu}(x_i) - \hat{\sigma}_i \hat{q} \mid x_i \in X_R, Ax_i \leq b\}$

if $\forall y \in Y^u,\ y > 0$ **then**

  $Y_R \leftarrow Y^l$

else

  $Y_R \leftarrow Y^u$

end

return X_R, Y_R